

编译原理·语法分析（二）：自顶向下分析

Syntax Analysis: Top-Down ——详细中文学习笔记

基于课件 4.Syntax_Analysis_Top_Down.pdf（中山大学·赵帅）整理

本笔记说明

本笔记覆盖第 4 讲《Syntax Analysis: Top-Down》共 89 页内容。主线是：
递归下降 → 左递归/回溯问题 → 预测分析 → FIRST/FOLLOW → LL(1) 分析表 → 表驱动 Parser
笔记按课件原顺序组织，所有核心概念给出中文解释、英文术语和页码标注，并在末尾附练习题与参考答案。

目录

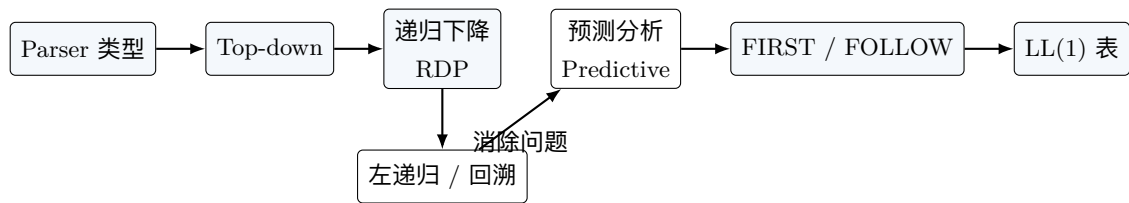
1 全局导图与 Parser 类型（第 3–8 页）	2
1.1 本讲主线 [p3,4]	2
1.2 自底向上分析 [p5]	3
1.3 自顶向下分析 [p6,7]	3
1.4 Top-down vs Bottom-up 示例 [p8]	4
2 递归下降分析与回溯（第 9–12 页）	4
2.1 递归下降分析的基本思想 [p10]	4
2.2 带回溯的 RDP [p11,12]	5
3 左递归问题与消除（第 13–23 页）	5
3.1 什么是左递归 [p13,14]	5
3.2 消除直接左递归：单个递归候选式 [p15]	6
3.3 消除直接左递归：多个候选式 [p16]	6
3.4 间接左递归及系统消除算法 [p17–19]	6
3.5 R/Q/S 示例 [p20–22]	7
3.6 递归下降分析的局限 [p23]	7
4 预测分析与左因子提取（第 24–29 页）	8
4.1 预测分析的定义 [p25,26]	8
4.2 共同前缀问题 [p27]	8
4.3 左因子提取 [p28]	8
4.4 预测分析的两个前置改写 [p29]	9

1 全局导图与 <i>PARSER</i> 类型 (第 3–8 页)	2
5 FIRST 与 FOLLOW (第 30–50 页)	9
5.1 FIRST 的定义与作用 [p30]	9
5.2 FOLLOW 的定义与作用 [p31,32]	9
5.3 FOLLOW 使用例子 [p32]	10
5.4 计算 FIRST(X) 的规则 [p33]	10
5.5 计算 FIRST(alpha) 的规则	10
5.6 表达式文法的 FIRST 示例 [p35–38]	11
5.7 计算 FOLLOW 的规则 [p39,49,50]	11
5.8 表达式文法的 FOLLOW 示例 [p40–44,50]	12
6 LL(1) 文法 (第 51–53 页)	12
6.1 LL(1) 的判定条件 [p51,52]	12
6.2 LL(1) 和 LL(k) 的含义 [p53]	13
7 LL(1) Parser 实现与非递归算法 (第 54–58 页)	13
7.1 递归 LL(1) Parser [p54]	13
7.2 非递归 LL(1) Parser 的组件 [p55,56]	13
7.3 非递归 LL(1) 分析算法 [p57,58]	14
8 使用 LL(1) 分析表 (第 59–77 页)	14
8.1 表达式文法与预测分析表 [p59]	14
8.2 完整分析过程 [p60–77]	15
9 构建 LL(1) 分析表 (第 78–81 页)	16
9.1 填表算法 [p78–80]	16
9.2 表达式文法的完整 FIRST/FOLLOW 和表 [p81]	16
10 判断 LL(1)、非 LL(1) 处理与复杂度 (第 82–89 页)	16
10.1 判断一个文法是否 LL(1) [p82]	16
10.2 非 LL(1) 文法怎么办 [p83]	17
10.3 LL(1) 时间与空间复杂度 [p84,85]	17
10.4 本讲总结 [p86–89]	17
11 练习题与参考答案	18

1 全局导图与 Parser 类型 (第 3–8 页)

1.1 本讲主线 [p3,4]

本讲研究自顶向下语法分析 (Top-down parsing)。课件的思维导图可以压缩为三条线：



- 第一部分讲语法分析器两大类型：**自顶向下** (Top-down) 与 **自底向上** (Bottom-up)。
- 第二部分讲自顶向下的通用形式：**递归下降分析** (Recursive-Descent Parsing, RDP)，以及它遇到的左递归和回溯问题。
- 第三部分讲无回溯的**预测分析** (Predictive Parsing)，核心工具是 FIRST、FOLLOW 和 LL(1) 分析表。

1.2 自底向上分析 [p5]

自底向上分析 (Bottom-up Parsing)

自底向上分析从输入串的叶子节点开始，逐步向分析树根部构造。它尝试把输入串不断**规约** (reduce) 为开始符号。

特点：

- 从叶子到根，方向与推导相反。
- 寻找**最右推导的逆过程** (reverse order of the rightmost derivation)，也称规范规约。
- Parser 代码结构通常不像文法，手写实现困难。
- 工具支持成熟，例如 Yacc、Bison。

1.3 自顶向下分析 [p6,7]

自顶向下分析 (Top-down Parsing)

自顶向下分析从分析树根部开始，按预定顺序向叶子扩展。它可以看作是在为输入串寻找一条**最左推导** (leftmost derivation)。

特点：

- 从开始符号出发，通常按深度优先、先根次序构造分析树。
- 每次选择最左边尚未展开的非终结符，所以目标是最左推导。
- 选定产生式后，把产生式右部的终结符与输入串匹配。
- Parser 代码结构接近文法，适合手写；也有 ANTLR 等自动工具。

自顶向下的关键问题

推导每一步必须做两个选择：替换哪个非终结符、对这个非终结符使用哪条产生式。自顶向下分析固定选择“最左非终结符”，剩下的关键问题就是：**如何选择正确产生式**。

1.4 Top-down vs Bottom-up 示例 [p8]

课件使用文法：

$$S \rightarrow AB, \quad A \rightarrow aA \mid a, \quad B \rightarrow bB \mid b$$

识别句子 $aabb$ 。

最左推导为：

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabb$$

自底向上分析则反向规约，例如先把第二个 a 规约为 A ，再把 aA 规约为 A ，再处理 bB ，最后规约为 S 。

对比项	自顶向下	自底向上
构造方向	根 → 叶子	叶子 → 根
推导关系	寻找最左推导	最右推导的逆过程
实现难度	代码近似文法，便于手写	更强但手写困难
常见工具	ANTLR	Yacc / Bison

2 递归下降分析与回溯（第 9–12 页）

2.1 递归下降分析的基本思想 [p10]

递归下降分析 (Recursive-Descent Parsing, RDP)

递归下降分析是自顶向下分析的一种通用形式。它为每个非终结符编写一个过程，执行从开始符号对应过程开始。如果过程体能扫描完整整个输入串，就宣布分析成功。

直观写法：

- 非终结符 A 对应函数 `parseA()`。
- 产生式 $A \rightarrow XYZ$ 对应函数体中依次处理 X, Y, Z 。
- 终结符直接与输入 `token` 匹配；非终结符则调用对应过程。

RDP 的伪代码本身可能是**不确定的** (nondeterministic)，因为当 A 有多个候选式时，伪代码没有指定应该选择哪一个。

2.2 带回溯的 RDP [p11,12]

回溯 (Backtracking)

当一个非终结符有多个候选式时，分析器按某种顺序试用候选式。如果当前候选式最终导致失配，就退回到之前的位置，恢复输入指针和分析树状态，再尝试下一个候选式。

匹配规则：

- 终结符与当前输入相同：匹配成功，输入指针前进。
- 终结符与当前输入不同：当前尝试失败，回溯或直接报错。
- 如果没有任何推导能生成完整输入串，分析失败。

课件例子：

$$G[S]: \quad S \rightarrow xAy, \quad A \rightarrow ** \mid *$$

输入为 $x * y$ 。如果先尝试 $A \rightarrow **$ ，读到第二个 $*$ 时失配，需要回溯；再尝试 $A \rightarrow *$ ，即可成功。

回溯的问题

回溯看起来简单，但代价很高：它会重复分析相同输入片段，最坏可达指数时间；同时已经加入分析树的节点还要撤销，实现麻烦。因此实际编译器更希望使用无回溯的预测分析。

3 左递归问题与消除 (第 13–23 页)

3.1 什么是左递归 [p13,14]

左递归 (Left Recursion)

如果文法中存在非终结符 A ，使得

$$A \Rightarrow^+ A\alpha$$

则该文法是左递归的。也就是说，从 A 出发经过一步或多步推导后，句型最左边又出现了 A 自己。

左递归会让自顶向下分析陷入无限循环。原因是自顶向下分析总是先展开最左非终结符；若 $A \rightarrow A\alpha$ ，展开 A 后最左边还是 A ，输入符号还没被消费，就会无限递归。

左递归分两类：

- **直接左递归** (Immediate Left Recursion)：存在产生式 $A \rightarrow A\alpha$ 。
- **间接左递归** (Non-immediate / Indirect Left Recursion)：经过两步或多步回到自身，例如 $A \rightarrow B\beta$, $B \rightarrow A\alpha$ 。

3.2 消除直接左递归：单个递归候选式 [p15]

基本形式：

$$A \rightarrow A\alpha \mid \beta$$

其中 β 不以 A 开头。它生成的串形如：

$$\beta\alpha\alpha\cdots\alpha$$

因此可以改写为右递归：

$$A \rightarrow \beta A', \quad A' \rightarrow \alpha A' \mid \varepsilon$$

通俗理解

左递归规则 $A \rightarrow A\alpha$ 的效果是“不断在右边追加 α ，最后用 β 收尾”。改写后先输出 β ，再由新非终结符 A' 决定追加多少个 α 。两者生成同一语言，但右递归不会让自顶向下分析卡在最左 A 上。

3.3 消除直接左递归：多个候选式 [p16]

一般形式：

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

其中没有任何 β_i 以 A 开头。改写为：

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon$$

课件练习中的经典表达式文法：

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid i$$

消除直接左递归后：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

3.4 间接左递归及系统消除算法 [p17-19]

课件例子：

$$S \rightarrow Qc \mid c, \quad Q \rightarrow Rb \mid b, \quad R \rightarrow Sa \mid a$$

单看每条产生式没有 $S \rightarrow S\alpha$ 这种直接形式，但存在：

$$S \Rightarrow Qc \Rightarrow Rbc \Rightarrow Sabc$$

所以 S 间接左递归， Q, R 也参与左递归环。

系统算法适用于无环、无 ε -产生式的文法：

1. 将非终结符按某个顺序排列为 $A_1, A_2, \dots, A_n$ 。
2. 对 $i = 1$ 到 n , 对每个 $j < i$, 把形如 $A_i \rightarrow A_j \gamma$ 的产生式替换为

$$A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$$

其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ 是当前 A_j 的所有产生式。

3. 对当前 A_i 消除直接左递归。
4. 最后删除从开始符号不可达的无效产生式。

这个算法本质上先把间接左递归展开为直接左递归, 再调用直接左递归消除公式。

3.5 R/Q/S 示例 [p20-22]

使用顺序 R, Q, S :

$$R \rightarrow Sa \mid a, \quad Q \rightarrow Rb \mid b, \quad S \rightarrow Qc \mid c$$

处理后得到:

$$Q \rightarrow Sab \mid ab \mid b$$

继续替换 $S \rightarrow Qc$:

$$S \rightarrow Sabc \mid abc \mid bc \mid c$$

此时 S 出现直接左递归。消除得到:

$$S \rightarrow abcS' \mid bcS' \mid cS'$$

$$S' \rightarrow abcS' \mid \varepsilon$$

最后 Q, R 从开始符号 S 不可达, 可删除。

如果步骤 1 选择顺序 S, Q, R , 最终文法形式会不同, 例如课件给出:

$$R \rightarrow bcaR' \mid caR' \mid aR', \quad R' \rightarrow bcaR' \mid \varepsilon$$

形式不同但语言等价。非终结符排序会影响输出形式, 但不改变等价性。

3.6 递归下降分析的局限 [p23]

递归下降分析简单、通用, 但不流行的原因主要是:

- 必须先消除左递归。
- 回溯会重复分析相同输入片段, 效率低。
- 穷尽所有可能候选式的尝试, 最坏可能指数时间。
- 回溯时需要撤销已经构造的 parse tree 节点, 实现复杂。

4 预测分析与左因子提取 (第 24–29 页)

4.1 预测分析的定义 [p25,26]

预测分析 (Predictive Parsing)

预测分析是无回溯的递归下降分析。它只根据当前正在处理的非终结符和向前看的固定数量输入符号，预测应使用哪条产生式。最常见情况只看 1 个输入符号。

选择产生式的依据：

- 当前要展开的非终结符 A 。
- 当前输入符号，也称**向前看符号** (lookahead symbol)。

例子：

$$S \rightarrow aBD \mid bBB, \quad B \rightarrow c \mid bce, \quad D \rightarrow d$$

读到第一个输入符号是 a 时选 $S \rightarrow aBD$ ，读到 b 时选 $S \rightarrow bBB$ ，无需回溯。若改成

$$S \rightarrow aBD \mid aBB$$

两个候选式前缀相同，只看一个 a 无法选择。

4.2 共同前缀问题 [p27]

仍看：

$$S \rightarrow xAy, \quad A \rightarrow ** \mid *$$

当要展开 A 且当前输入为 $*$ 时，两个候选式都可能匹配。共同前缀会让预测分析无法立即选择，导致回溯风险。

4.3 左因子提取 [p28]

左因子提取 (Left Factoring)

左因子提取把多个候选式的公共前缀提出来，把选择推迟到读到足够多输入符号之后。

通用形式：

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \cdots \mid \alpha\beta_n \mid \gamma$$

其中 $\alpha \neq \varepsilon$ 是公共前缀， γ 表示不以 α 开头的其他候选式。改写为：

$$A \rightarrow \alpha A' \mid \gamma, \quad A' \rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

重复应用直到同一非终结符的候选式不再共享公共前缀。

例子：

$$S \rightarrow abc \mid abd \mid ae$$

第一步：

$$S \rightarrow aA', \quad A' \rightarrow bc \mid bd \mid e$$

第二步：

$$S \rightarrow aA', \quad A' \rightarrow bB' \mid e, \quad B' \rightarrow c \mid d$$

4.4 预测分析的两个前置改写 [p29]

阻碍预测分析的典型文法模式：

- 左递归：lookahead 只有在匹配终结符时才前进，左递归会无限展开。
- 共同前缀：多个候选式看起来一样，必须回溯或推迟选择。

对应处理：

左递归 $\rightarrow$ 消除左递归 共同前缀 $\rightarrow$ 左因子提取

但课件强调：仅做这两步还不一定完全够，还需要 FIRST 和 FOLLOW 来判断什么时候能无回溯选择产生式。

5 FIRST 与 FOLLOW (第 30–50 页)

5.1 FIRST 的定义与作用 [p30]

FIRST 集 (FIRST Set / 终结首符集)

对任意文法符号串 α ， $\text{FIRST}(\alpha)$ 是所有能出现在 α 推导结果开头的终结符集合：

$$\text{FIRST}(\alpha) = \{a \mid \alpha \Rightarrow^* a \cdots, a \in V_T\}$$

如果 $\alpha \Rightarrow^* \varepsilon$ ，则 $\varepsilon \in \text{FIRST}(\alpha)$ 。

作用：若 $A \rightarrow \alpha \mid \beta$ ，且

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$$

那么看下一个输入符号 a 就能决定选择哪个候选式，因为 a 不可能同时属于两个 FIRST 集。

5.2 FOLLOW 的定义与作用 [p31,32]

FOLLOW 集 (FOLLOW Set / 后继终结符号集)

对非终结符 A ， $\text{FOLLOW}(A)$ 是有可能紧跟在 A 右侧的终结符集合：

$$\text{FOLLOW}(A) = \{a \mid S \Rightarrow^* \cdots Aa \cdots, a \in V_T\}$$

如果 A 能出现在某个句型最右端，则结束标记 $\$$ 属于 $\text{FOLLOW}(A)$ 。

FIRST 和 FOLLOW 的区别：

- $\text{FIRST}(\alpha)$: 看 α 自己展开后最前面的终结符。
- $\text{FOLLOW}(A)$: 看 A 这个非终结符后面可能紧跟什么终结符, 不包含 A 自己展开出来的符号。
如果当前要展开 A , 输入符号 a 不在任何候选式的 FIRST 中, 但某个候选式 α_i 可以推出空串:

$$\varepsilon \in \text{FIRST}(\alpha_i)$$

并且

$$a \in \text{FOLLOW}(A)$$

则应选择 $A \rightarrow \alpha_i$ 。意思是: A 可以不产生任何东西, 让后面的符号去匹配当前输入。

5.3 FOLLOW 使用例子 [p32]

文法:

$$S \rightarrow aA \mid d, \quad A \rightarrow bAS \mid \varepsilon$$

输入 abd 。相关集合:

$$\text{FIRST}(aA) = \{a\}, \quad \text{FIRST}(d) = \{d\}, \quad \text{FIRST}(bAS) = \{b\}$$

$$\text{FIRST}(A) = \{b, \varepsilon\}, \quad \text{FIRST}(S) = \{a, d\}, \quad \text{FOLLOW}(A) = \{\$, a, d\}$$

推导过程:

$$S \Rightarrow aA \Rightarrow abAS$$

此时输入还剩 d , 当前 A 的候选式 bAS 不能以 d 开头, 但 $A \rightarrow \varepsilon$ 可用, 并且 $d \in \text{FOLLOW}(A)$, 所以选择 $A \rightarrow \varepsilon$, 继续得到:

$$abAS \Rightarrow abS \Rightarrow abd$$

5.4 计算 $\text{FIRST}(X)$ 的规则 [p33]

对所有文法符号 X 反复应用以下规则, 直到没有新终结符或 ε 可加入:

1. 若 $X \in V_T$, 则 $\text{FIRST}(X) = \{X\}$ 。
2. 若 $X \in V_N$, 且存在 $X \rightarrow \varepsilon$, 则把 ε 加入 $\text{FIRST}(X)$ 。
3. 若 $X \in V_N$, 且存在 $X \rightarrow Y_1 Y_2 \cdots Y_k$, 则:
 - 将 $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$ 加入 $\text{FIRST}(X)$ 。
 - 如果 $Y_1 \Rightarrow^* \varepsilon$, 继续加入 $\text{FIRST}(Y_2) \setminus \{\varepsilon\}$, 以此类推。
 - 若所有 Y_i 都能推出 ε , 则把 ε 加入 $\text{FIRST}(X)$ 。

5.5 计算 $\text{FIRST}(\alpha)$ 的规则 [p34,46-48]

对符号串 $\alpha = X_1 X_2 \cdots X_n$:

1. 把 $\text{FIRST}(X_1)$ 中所有非 ε 符号加入 $\text{FIRST}(\alpha)$ 。
2. 如果 $X_1, \dots, X_{i-1}$ 都能推出 ε , 则把 $\text{FIRST}(X_i) \setminus \{\varepsilon\}$ 加入 $\text{FIRST}(\alpha)$ 。
3. 如果所有 $X_1, \dots, X_n$ 都能推出 ε , 则把 ε 加入 $\text{FIRST}(\alpha)$ 。

FIRST 计算的直觉

FIRST 的本质是问“这个符号或符号串最终生成的句子，第一个终结符可能是什么”。如果前面的符号可能消失为空串，就继续看后一个符号。

5.6 表达式文法的 FIRST 示例 [p35–38]

文法：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid id$$

最终 FIRST 集为：

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

计算需要迭代：第一次可能只能得出 E' 、 T' 、 F 的集合；再由 F 推到 T ，再由 T 推到 E 。每轮应用规则后都要检查集合是否变化，直到不再变化。

5.7 计算 FOLLOW 的规则 [p39,49,50]

对所有非终结符 A ，反复应用以下规则，直到没有新符号可加入：

1. 将 $\$$ 放入开始符号 S 的 FOLLOW 集。
2. 若存在产生式 $A \rightarrow \alpha B \beta$ ，则把 $\text{FIRST}(\beta) \setminus \{\varepsilon\}$ 加入 $\text{FOLLOW}(B)$ 。
3. 若存在产生式 $A \rightarrow \alpha B$ ，或 $A \rightarrow \alpha B \beta$ 且 $\varepsilon \in \text{FIRST}(\beta)$ ，则把 $\text{FOLLOW}(A)$ 加入 $\text{FOLLOW}(B)$ 。

FOLLOW 计算的直觉

若 B 后面直接跟着 β ，那么 β 可能开头的终结符也可能紧跟 B 。若 β 可以消失，或 B 已经在产生式末尾，那么 B 后面会接上左部非终结符 A 后面能出现的符号。

5.8 表达式文法的 FOLLOW 示例 [p40–44,50]

仍使用表达式文法，最终 FOLLOW 集为：

$$\text{FOLLOW}(E) = \{\$, \,)\}$$

$$\text{FOLLOW}(T) = \{+, \$, \,)\}$$

$$\text{FOLLOW}(E') = \{\$, \,)\}$$

$$\text{FOLLOW}(T') = \{+, \$, \,)\}$$

$$\text{FOLLOW}(F) = \{*, +, \$, \,)\}$$

关键来源：

- E 是开始符号，所以 $\$ \in \text{FOLLOW}(E)$ 。
- $F \rightarrow (E)$ ，所以 $) \in \text{FOLLOW}(E)$ 。
- $E \rightarrow TE'$ ： $+$ $\in \text{FIRST}(E')$ ，所以 $+$ $\in \text{FOLLOW}(T)$ ；又因为 $E' \Rightarrow \varepsilon$ ，所以 $\text{FOLLOW}(E) \subseteq \text{FOLLOW}(T)$ 。
- $T \rightarrow FT'$ ： $*$ $\in \text{FIRST}(T')$ ，所以 $*$ $\in \text{FOLLOW}(F)$ ；又因为 $T' \Rightarrow \varepsilon$ ，所以 $\text{FOLLOW}(T) \subseteq \text{FOLLOW}(F)$ 。

6 LL(1) 文法 (第 51–53 页)

6.1 LL(1) 的判定条件 [p51,52]

LL(1) 文法

能构造无回溯预测分析器的文法类称为 LL(1) 文法。对任意同一左部的两个不同产生式

$$A \rightarrow \alpha \mid \beta$$

必须满足下面条件。

-
1. 不存在某个终结符 a ，使得 α 和 β 都能推出以 a 开头的串。等价地：

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$$

2. α 和 β 至多一个能推出空串。

3. 如果 $\beta \Rightarrow^* \varepsilon$ ，则 α 不能推出以 $\text{FOLLOW}(A)$ 中任何终结符开头的串。等价地：

$$\varepsilon \in \text{FIRST}(\beta) \Rightarrow \text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$$

对 $\alpha \Rightarrow^* \varepsilon$ 的情况同理。

LL(1) 的重要性质

LL(1) 文法不含左递归；LL(1) 文法无二义性。但反过来不成立：无左递归、无二义的文法不一定是 LL(1)。

例子：

$$S \rightarrow EBD \mid FBB, \quad E \rightarrow a \cdots, \quad F \rightarrow a \cdots$$

由于两个候选式都可能以 a 开头，只看一个 lookahead 不能选择，因此不是 LL(1)。

6.2 LL(1) 和 LL(k) 的含义 [p53]

- 第一个 L ：从左到右扫描输入 (Left-to-right scanning)。
- 第二个 L ：产生最左推导 (Leftmost derivation)。
- 1：每步只看 1 个输入符号做决策。
- LL(k)：每步看 k 个输入符号做决策。

LL(0) 几乎没有实用价值。因为不用任何 lookahead 就能预测，意味着每个非终结符只能有一条规则，语言基本只能生成一个固定字符串。

7 LL(1) Parser 实现与非递归算法 (第 54–58 页)

7.1 递归 LL(1) Parser [p54]

以文法

$$S \rightarrow A \mid B, \quad A \rightarrow a, \quad B \rightarrow b$$

为例，递归 LL(1) parser 可以写成类似：

```
token = Next();
if token == a then A();
else if token == b then B();
else error;
```

递归实现隐式维护调用栈。这个栈本质上保存尚未完成的推导结果。

7.2 非递归 LL(1) Parser 的组件 [p55,56]

非递归实现使用显式栈和预测分析表。

组件	作用
输入缓冲区	保存待分析 token 串，并在末尾追加结束标记\$。
栈	保存尚未匹配或尚未展开的文法符号；底部也放\$。

预测分析表 $M[A, a]$	行是非终结符 A , 列是终结符或 $\$$ 。单元格保存应使用的产生式或 error。
预测分析程序	根据栈顶符号 X 和当前输入符号 a 执行动作。

7.3 非递归 LL(1) 分析算法 [p57,58]

初始状态:

$$\text{Input} = w\$, \quad \text{Stack} = S\$$$

设 X 是栈顶符号, a 是当前输入符号。

1. 如果 $X \in V_T$:

- 若 $X = a = \$$, 分析成功。
- 若 $X = a \neq \$$, 弹出 X , 输入指针前进。
- 若 $X \neq a$, 报语法错误。

2. 如果 $X \in V_N$:

- 若 $M[X, a]$ 中有产生式 $X \rightarrow Y_1 Y_2 \cdots Y_k$, 弹出 X , 并将右部逆序压栈: 先压 Y_k , 最后压 Y_1 , 使 Y_1 位于栈顶。
- 若 $M[X, a]$ 为空, 报语法错误。
- 若产生式是 $X \rightarrow \varepsilon$, 只弹出 X , 不压入任何符号。

为什么右部要逆序入栈

栈是后进先出。若产生式为 $X \rightarrow +TE'$, 希望下一步先处理 $+$, 再处理 T , 最后处理 E' 。因此压栈顺序必须是 E' 、 T 、 $+$ 。

8 使用 LL(1) 分析表 (第 59–77 页)

8.1 表达式文法与预测分析表 [p59]

课件使用表达式文法识别:

$$id + id * id$$

文法为:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid id$$

预测分析表为：

	<i>id</i>	+	*	(	)	\$
<i>E</i>	$E \rightarrow TE'$			$E \rightarrow TE'$		
<i>E'</i>	$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon \quad E' \rightarrow \varepsilon$		
<i>T</i>	$T \rightarrow FT'$			$T \rightarrow FT'$		
<i>T'</i>	$T' \rightarrow \varepsilon \quad T' \rightarrow *FT'$			$T' \rightarrow \varepsilon \quad T' \rightarrow \varepsilon$		
<i>F</i>	$F \rightarrow id$			$F \rightarrow (E)$		

8.2 完整分析过程 [p60–77]

输入缓冲区：

*id + id * id*\$

初始栈：

E\$

课件逐页演示的栈变化可以整理为下表。

已匹配	栈	剩余输入	动作
ε	<i>E</i> \$	<i>id + id * id</i> \$	$E \rightarrow TE'$
ε	<i>TE'</i> \$	<i>id + id * id</i> \$	$T \rightarrow FT'$
ε	<i>FT'E'</i> \$	<i>id + id * id</i> \$	$F \rightarrow id$
ε	<i>idT'E'</i> \$	<i>id + id * id</i> \$	匹配 <i>id</i>
<i>id</i>	<i>T'E'</i> \$	<i>+id * id</i> \$	$T' \rightarrow \varepsilon$
<i>id</i>	<i>E'</i> \$	<i>+id * id</i> \$	$E' \rightarrow +TE'$
<i>id</i>	<i>+TE'</i> \$	<i>+id * id</i> \$	匹配+
<i>id+</i>	<i>TE'</i> \$	<i>id * id</i> \$	$T \rightarrow FT'$
<i>id+</i>	<i>FT'E'</i> \$	<i>id * id</i> \$	$F \rightarrow id$
<i>id+</i>	<i>idT'E'</i> \$	<i>id * id</i> \$	匹配 <i>id</i>
<i>id + id</i>	<i>T'E'</i> \$	<i>*id</i> \$	$T' \rightarrow *FT'$
<i>id + id</i>	<i>*FT'E'</i> \$	<i>*id</i> \$	匹配*
<i>id + id*</i>	<i>FT'E'</i> \$	<i>id</i> \$	$F \rightarrow id$
<i>id + id*</i>	<i>idT'E'</i> \$	<i>id</i> \$	匹配 <i>id</i>
<i>id + id * id</i>	<i>T'E'</i> \$	\$	$T' \rightarrow \varepsilon$
<i>id + id * id</i>	<i>E'</i> \$	\$	$E' \rightarrow \varepsilon$
<i>id + id * id</i>	\$	\$	ACCEPT

这个过程模拟了最左推导。每次栈顶是非终结符，就查表展开；每次栈顶是终结符，就与输入匹配。

9 构建 LL(1) 分析表 (第 78–81 页)

9.1 填表算法 [p78–80]

LL(1) 分析表构建算法

对文法中每条产生式 $A \rightarrow \alpha$:

1. 对每个终结符 $a \in \text{FIRST}(\alpha)$, 把 $A \rightarrow \alpha$ 填入 $M[A, a]$ 。
2. 如果 $\varepsilon \in \text{FIRST}(\alpha)$, 则对每个 $b \in \text{FOLLOW}(A)$, 把 $A \rightarrow \alpha$ 填入 $M[A, b]$ 。
3. 如果 $\varepsilon \in \text{FIRST}(\alpha)$ 且 $\$ \in \text{FOLLOW}(A)$, 也把 $A \rightarrow \alpha$ 填入 $M[A, \$]$ 。

直观解释:

- 如果当前输入 a 可能是 α 展开结果的第一个符号, 就选择 $A \rightarrow \alpha$ 。
- 如果 α 能推出空串, 那它可以“什么都不生成”, 此时应看 A 后面允许出现什么, 即 $\text{FOLLOW}(A)$ 。

9.2 表达式文法的完整 FIRST/FOLLOW 和表 [p81]

符号	FIRST	FOLLOW
E	$\{ (, id \}$	$\{ \$,) \}$
E'	$\{ +, \varepsilon \}$	$\{ \$,) \}$
T	$\{ (, id \}$	$\{ +, \$,) \}$
T'	$\{ *, \varepsilon \}$	$\{ +, \$,) \}$
F	$\{ (, id \}$	$\{ *, +, \$,) \}$

由这些集合可得到前面使用的预测分析表。注意:

- $E' \rightarrow \varepsilon$ 填到 $M[E',)]$ 和 $M[E', \$]$, 因为 $\text{FOLLOW}(E') = \{ \$,) \}$ 。
- $T' \rightarrow \varepsilon$ 填到 $M[T', +]$ 、 $M[T',)]$ 、 $M[T', \$]$, 因为这些符号都在 $\text{FOLLOW}(T')$ 中。

10 判断 LL(1)、非 LL(1) 处理与复杂度 (第 82–89 页)

10.1 判断一个文法是否 LL(1) [p82]

两种方法:

1. 构建 LL(1) 分析表。如果每个表项至多一条产生式, 则文法是 LL(1); 若某个表项有多条规则冲突, 则不是 LL(1)。
2. 直接检查每个同左部的候选式 $A \rightarrow \alpha \mid \beta$:

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$$

若 $\varepsilon \in \text{FIRST}(\beta)$, 还需:

$$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$$

对 α 能推出空串的情况同理。

10.2 非 LL(1) 文法怎么办 [p83]

若文法不是 LL(1), 分两种情况:

- **语言本身仍可能是 LL(1):** 尝试改写文法, 例如消除左递归、提取左因子、消除二义性。
- **语言可能不是 LL(1):** 无法在文法层面完全消除冲突。此时可以人工指定冲突表项选择哪条规则, 前提是该选择语义上总是正确; 否则应使用更强 parser, 例如 LL(k) 或 LR(1)。

10.3 LL(1) 时间与空间复杂度 [p84,85]

- **时间复杂度:** 相对于输入长度是线性的。每个输入符号会在常数数量级步骤内被消费; 栈顶为终结符时一步匹配, 栈顶为非终结符时查表展开。LL(1) 无左递归, 不会在不消费输入的情况下无限应用同一规则。
- **空间复杂度:** 栈保存未匹配的推导片段, 通常小于输入长度。去掉 $X \rightarrow \varepsilon$ 后, 产生式右部长度不小于左部, 推导串单调扩展, 栈只是最终推导串的未匹配部分。
- **LL(k) 表大小:**

$$O(|N| \cdot |T|^k)$$

其中 N 是非终结符数量, T 是终结符数量。 k 增大时表规模指数增长, 这限制了 LL(2)、LL(3) 等在实践中的广泛使用。

10.4 本讲总结 [p86–89]

本讲需要掌握:

- Top-down parsing 与 Bottom-up parsing 的方向和推导关系。
- RDP、回溯以及回溯低效的原因。
- 直接/间接左递归的识别和消除。
- 共同前缀问题与左因子提取。
- FIRST、FOLLOW 的定义、计算规则和用途。
- LL(1)/LL(k) 文法定义和判定条件。
- 递归与非递归 LL(1) parser 实现。
- LL(1) 分析表的使用、构建和复杂度。

延伸阅读: Dragon Book 4.1.2、4.4.1、4.4.2、4.4.3–4.4.5。课件建议可跳读 4.1.3–4.1.4 的错误恢复, 以及 4.2.7 的 CFG 与正则表达式差异。

11 练习题与参考答案

练习 1: Top-down 与 Bottom-up

给定文法 $S \rightarrow AB$, $A \rightarrow aA \mid a$, $B \rightarrow bB \mid b$, 写出 $aabb$ 的最左推导, 并说明自底向上分析与它的关系。

答案:

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabb$$

自底向上分析从 $aabb$ 出发做规约, 方向与推导相反, 可看作最右推导的逆过程。

练习 2: 判断直接左递归并消除

消除文法 $A \rightarrow Aa \mid b$ 的直接左递归。

答案: 套用 $A \rightarrow A\alpha \mid \beta$:

$$A \rightarrow bA', \quad A' \rightarrow aA' \mid \varepsilon$$

它生成 ba^n ($n \geq 0$)。

练习 3: 多个候选式左递归

消除:

$$A \rightarrow Aa \mid Ab \mid c \mid d$$

答案:

$$A \rightarrow cA' \mid dA'$$

$$A' \rightarrow aA' \mid bA' \mid \varepsilon$$

练习 4: 左因子提取

对文法

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid \text{other}$$

提取左因子。

答案: 公共前缀是 $\text{if } E \text{ then } S$, 改写为:

$$S \rightarrow \text{if } E \text{ then } S S' \mid \text{other}$$

$$S' \rightarrow \text{else } S \mid \varepsilon$$

练习 5: 计算 FIRST

文法:

$$S \rightarrow AB, \quad A \rightarrow a \mid \varepsilon, \quad B \rightarrow b$$

求 $\text{FIRST}(S)$ 、 $\text{FIRST}(A)$ 、 $\text{FIRST}(B)$ 。

答案:

$$\text{FIRST}(A) = \{a, \varepsilon\}, \quad \text{FIRST}(B) = \{b\}$$

由于 A 可为空, 所以 $S = AB$ 的开头既可能来自 A , 也可能来自 B :

$$\text{FIRST}(S) = \{a, b\}$$

练习 6: 计算 FOLLOW

文法:

$$S \rightarrow AB, \quad A \rightarrow a \mid \varepsilon, \quad B \rightarrow b$$

求 $\text{FOLLOW}(S)$ 、 $\text{FOLLOW}(A)$ 、 $\text{FOLLOW}(B)$ 。

答案: 因为 S 是开始符号:

$$\text{FOLLOW}(S) = \{\$ \}$$

在 $S \rightarrow AB$ 中, A 后面是 B , 所以:

$$\text{FOLLOW}(A) = \text{FIRST}(B) = \{b\}$$

B 在产生式末尾, 所以:

$$\text{FOLLOW}(B) = \text{FOLLOW}(S) = \{\$ \}$$

练习 7: 判断 LL(1)

文法:

$$S \rightarrow aA \mid bB, \quad A \rightarrow c, \quad B \rightarrow c$$

是否是 LL(1)?

答案: 是。对 S 的两个候选式:

$$\text{FIRST}(aA) = \{a\}, \quad \text{FIRST}(bB) = \{b\}$$

两者不相交; A, B 各只有一条产生式。因此可用一个 lookahead 决策。

练习 8: 判断非 LL(1)

文法:

$$S \rightarrow aA \mid aB, \quad A \rightarrow c, \quad B \rightarrow d$$

是否是 LL(1)? 如何改写?

答案: 不是 LL(1), 因为两个 S 候选式 FIRST 都是 $\{a\}$ 。提取左因子:

$$S \rightarrow aS', \quad S' \rightarrow A \mid B, \quad A \rightarrow c, \quad B \rightarrow d$$

若继续替换, 也可写为:

$$S \rightarrow aS', \quad S' \rightarrow c \mid d$$

此时是 LL(1)。

练习 9：构造分析表项

表达式文法中：

$$E' \rightarrow +TE' \mid \varepsilon, \quad \text{FOLLOW}(E') = \{\$, \,)\}$$

应如何填写 E' 这一行的 LL(1) 表？

答案：因为 $\text{FIRST}(+TE') = \{+\}$ ，所以：

$$M[E', +] = E' \rightarrow +TE'$$

因为 $\varepsilon \in \text{FIRST}(\varepsilon)$ ，应在 $\text{FOLLOW}(E')$ 对应列填写空产生式：

$$M[E', \$] = E' \rightarrow \varepsilon, \quad M[E',)] = E' \rightarrow \varepsilon$$

练习 10：非递归 LL(1) 动作

若栈顶是非终结符 T' ，当前输入符号是 $*$ ，且分析表有

$$M[T', *] = T' \rightarrow *FT'$$

非递归 LL(1) parser 应如何处理？

答案：弹出 T' ，再把产生式右部逆序压栈：先压 T' ，再压 F ，最后压 $*$ 。这样下一步栈顶是 $*$ ，可以先与输入符号匹配。

练习 11：复杂度

为什么 LL(1) 分析通常是线性时间？

答案：每个输入符号最终只会被匹配并消费一次。栈顶是终结符时一步匹配；栈顶是非终结符时查表展开。LL(1) 无左递归，不会无限展开而不消费输入，因此总步骤数与输入长度成线性关系。

考前速记

- Top-down：根到叶，寻找最左推导；Bottom-up：叶到根，最右推导逆过程。
- RDP：每个非终结符一个过程；问题是左递归和回溯。
- 直接左递归： $A \rightarrow A\alpha \mid \beta$ 改为 $A \rightarrow \beta A'$ ， $A' \rightarrow \alpha A' \mid \varepsilon$ 。
- 左因子： $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ 改为 $A \rightarrow \alpha A'$ ， $A' \rightarrow \beta_1 \mid \beta_2$ 。
- FIRST 看“自己展开后的第一个终结符”；FOLLOW 看“非终结符右边可能紧跟什么”。
- LL(1)：左到右扫描、最左推导、1 个 lookahead。
- LL(1) 表项冲突说明文法不是 LL(1)。
- 构表：FIRST 填普通产生式；若能推出 ε ，按 FOLLOW 填空产生式。
- 非递归 LL(1)：输入缓冲区 + 显式栈 + 分析表；产生式右部逆序入栈。